Tracking the Lifecycle of Kubernetes Configuration Smells Across Versions: Baseline, Artifacts, and Related Work (Milestone 2)

Group 4
ENGR 5570G Software Maintenance and Evolution, Ontario Tech University
Rohan Muslekar (101006689), Ankit Suman (101041764), Yasaminzahra Valitabar (101001242),
Bayan Sayyadifar (101035809), Seyedeh Rojan Razavi (101001141)

***Abstract*: We report the working foundation of our study of how Kubernetes configuration smells persist, disappear, or reappear across released versions. A rerunnable pipeline runs two static analyzers, kube-linter and kube-score, over 7 open-source projects at five release tags each (35 versions, all valid). It tracks 862 smell instances, counted per analyzer, across the tags. Smells are sticky: 82.3% of the instances present at a project's first sampled tag are still present at its last, and none reappeared. Most removals (73.5%) happened because the object itself was deleted, largely unrendered kustomize patch fragments in one flux2 directory, not because the smell was fixed. Only 30 removals are fixes once a known analyzer artifact is set aside. On object–category pairs, the two analyzers agree on 55.6%, or 78.5% when hygiene, which only kube-linter checks, is left out.**

I. Study Context and Related Work

Questions. RQ1 (prevalence, supporting) asks which smell categories occur and how often. RQ2 (lifecycle, primary) asks whether individual smell instances persist, disappear, or reappear between released versions. RQ3 (explanation, primary) traces representative transitions to the change that caused them. This milestone delivers RQ1 and a first RQ2 result, plus a manual check that prepares RQ3. RQ4 (image CVEs) remains an optional stretch goal.

Systems and versions. Table I lists the 7 study systems. A candidate is included when it: (i) ships plain Kubernetes YAML in-tree; (ii) had commits in the last six months and has at least 500 stars; and (iii) has at least five release tags spanning at least 12 months.

4 candidates failed and are recorded with their reason in `data/repos.json`. ingress-nginx is archived. cert-manager and linkerd2 ship only Helm charts. opentelemetry-demo removed its in-tree YAML.

For each system we take the latest patch release of five minor versions, evenly spaced over the minors first released since 2022-10-22, with the newest included. This gives three to four years of history per project, so the lifecycle has room to show. The systems cover an application (microservices-demo), monitoring (kube-prometheus), GitOps (argo-cd, flux2), a service-mesh sample (istio Bookinfo), serverless (Knative serving), and CI (Tekton pipeline).

How the literature shapes our choices. *Categories.* Rahman et al. [1] (TOSEM 2023) mined 2,039 Kubernetes manifests from 92 repositories and validated 11 security-misconfiguration categories. These seed our security-and-privilege category, for example absent securityContext and host namespace access. *Smell framing.* Sharma et al. [2] catalogued configuration smells in 4,621 Puppet repositories, and Rahman et al. [3] defined security smells for infrastructure-as-code scripts. Together they justify treating analyzer warnings on configuration as smells. They also motivate keeping security, reliability, and hygiene smells apart rather than counting all warnings together. *Lifecycle angle.* Zerouali et al. [4] studied 9,482 Helm charts and found outdated images and known vulnerabilities to be widespread. That view is cross-sectional and at chart level; we follow individual smell instances within a project across its releases, which is the gap RQ2 targets. *Mapping discipline.* Oliveira et al. [5] replicated an IaC defect taxonomy on 3,364 defect-related commits from 285 repositories and found that its categories generalize. We follow the same discipline: every analyzer check is mapped to a fixed category with a written rationale (`data/check_categories.csv`), instead of using ad-hoc labels. *Two analyzers, overlap reported.* Lenarduzzi et al. [6] found that six static analysis tools rarely agree on the same issues and have low precision. We therefore run two independent analyzers and report their overlap *before* any de-duplication (Section III), as requested in the Milestone 1 feedback.

II. Data and Artifacts

Sources and revisions. The data source is each system's public git repository at the selected tags. `data/versions.csv` records the full commit SHA and commit date of all 35 tags. The tags were selected on 2026-10-22 by `python3 -m kst.select_tags`, which returns the same tags until the projects publish newer releases.

Collection and cleaning. `scripts/collect.sh` does the following for each tag:

- checks out the tag;
- keeps only YAML files that are plain Kubernetes objects. Kustomization files, Helm-templated files and non-Kubernetes YAML are skipped, and all 217 skipped files are logged with their reason;
- records an inventory of the objects defined at that tag;

TABLE I
STUDY SYSTEMS: MANIFEST PATH AND FIRST/LAST SAMPLED TAG (YEAR)

System	Manifest path	First tag	Last tag
microservices-demo	`kubernetes-manifests`	v0.5.2 (2023)	v0.10.7 (2026)
kube-prometheus	`manifests`	v0.12.0 (2023)	v0.19.0 (2026)
argo-cd	`manifests/install.yaml`	v2.5.22 (2023)	v3.5.6 (2026)
flux2	`manifests`	v0.36.0 (2022)	v2.9.6 (2026)
istio	`samples/bookinfo/platform/kube`	1.16.7 (2023)	1.31.2 (2026)
serving	`config/core`	v0.36.4 (2023)	v0.50.0 (2026)
pipeline	`config`	v0.41.3 (2023)	v1.17.0 (2026)

- runs kube-linter v0.8.3 [7] (all built-in checks) and kube-score v1.20.0 [8].

File paths are stored relative to the repository root. A tag is marked invalid and left out of tracking when either analyzer's output is empty or not JSON (kube-score prints nothing when it crashes) or when it has no manifests, so a crashed run does not look like removed smells. kube-linter skips files it cannot load without any message, so we also lint each file alone to find them. 3 files fail this way, all `List` files in kube-prometheus, at every tag; they are logged and not analyzed. Our proposal's fourth criterion (no analyzer parse errors) would have excluded kube-prometheus for these files alone, so we report them instead.

The normalizer turns the 19,346 raw reports into findings. A kube-score check counts when it is graded below 10 and not skipped. All 62 checks observed are mapped to security, resource, image, hygiene, or *other*. The 7 *other* checks are excluded from the smell analysis: style and opinionated defaults such as YAML key order, and kube-linter's two example ownership policies (an owner label, an email annotation), which fire on almost every object.

Environment. Fedora Linux 44, Go 1.26.9 (builds the pinned analyzers into `tools/bin`), Python 3.14.8 using only the standard library, git 2.56, jq 1.8.1. The method uses no LLMs.

Artifact. All scripts, raw analyzer output, the object inventories, the category map, the derived tables, and the manual spot-check are in https://github.com/Rohan-Muslekar/kube-smell-tracker (branch `master`). `./tools/install.sh && make all` rebuilds everything.

We verified reproduction from a fresh clone on two machines (Fedora, and Ubuntu under WSL). Both times the derived tables and `versions.csv` were byte-identical. The raw analyzer files differed in kube-linter's run timestamp and the order of kube-score's checks, and on WSL in a few more reports that we are still comparing; none of these changed the derived tables. 83 unit and pipeline tests cover parsing, identity, transition labelling and its edge cases, output validation, and the analyzer wrapper. CI reruns them and the analysis on every change.

III. WORKING BASELINE

Instance identity and transitions. A smell instance is keyed by (project, tool, kind, namespace, name, check). For each pair of consecutive sampled tags, an instance is *persisted*, *introduced*, *removed*, or *reappeared* (present again after an earlier removal).

TABLE II
SMELL INSTANCES AND TRANSITIONS PER CATEGORY AND TOOL (KL = KUBE-LINTER, KS = KUBE-SCORE). INST. AND BASE COUNT INSTANCES (ALL, AND PRESENT AT THE FIRST TAG); THE OTHER COLUMNS COUNT TAG-TO-TAG STEPS. PARENTHESES: INTRODUCTIONS WITH A NEW OBJECT; REMOVALS WITH THE OBJECT DELETED.

Category	Tool	Inst.	Base	Persist	Introd.	Removed	Reapp.
Security & privilege	KL	147	126	509	21 (5)	39 (22)	0
Security & privilege	KS	114	111	392	3 (2)	25 (16)	0
Resource & reliability	KL	216	212	808	4 (4)	28 (26)	0
Resource & reliability	KS	204	199	745	5 (3)	33 (22)	0
Image & supply chain	KL	25	24	87	1 (1)	6 (6)	0
Image & supply chain	KS	51	48	182	3 (1)	10 (10)	0
Hygiene	KL	95	91	366	4 (0)	4 (4)	0
Hygiene	KS	10	10	38	0 (0)	2 (2)	0
Total		862	821	3127	41 (16)	147 (108)	0

The object inventory then splits two of these labels:

- a removal whose object no longer exists is an *object deletion*, not a fix;
- an introduction that arrives together with a new object is counted separately.

The inventory matched every object either analyzer reported. Cluster-scoped kinds such as ClusterRole always get an empty namespace, since Kubernetes ignores one there.

Observations (RQ1, RQ2). Table II reports the results across 35 versions: 8,711 distinct findings and 862 smell instances (483 kube-linter, 379 kube-score). Instances are per tool, so a problem both analyzers flag counts once for each; together they fall on 276 distinct object–category pairs.

- **Prevalence.** Resource-and-reliability smells are the most frequent for both tools, followed by security.
- **Persistence.** Smells are persistent: 676 of the 821 instances present at a project's first sampled tag (82.3%; 83.2% for kube-linter, 81.2% for kube-score) are still present at its last, three to four years later. Every one of those survivors is present at all five tags.
- **Removals.** Of 147 removals, 108 (73.5%) came from deleting the object. 84 of these come from flux2 dropping its credential-sync integrations, which are unrendered kustomize patch fragments. 39 removals happened while the object stayed, but 9 of them are the kube-score probe

TABLE III
ANALYZER OVERLAP BEFORE DE-DUPLICATION: DISTINCT OBJECT–CATEGORY PAIRS OVER ALL TAGS, ON THE OBJECT KINDS KUBE-SCORE SCORES

Category	kube-linter only	kube-score only	Both
Security & privilege	1	7	45
Resource & reliability	5	0	54
Image & supply chain	0	21	25
Hygiene	73	0	10

TABLE IV
SPOT-CHECK OUTCOME BY STRATUM (CAUSE, COUNT)

Stratum	Verified cause
persisted	persisting smell (5)
fixed	real fix (4); analyzer artifact (1)
object deleted	object deleted (5)
introduced	side effect of fix (3); new object (2)

artifact described below. That leaves 30 fixes over 5 systems, or 24 when a fix that both tools see on the same object, category, and step counts once.

- **Introductions.** 16 of the 41 introductions arrived with a new object, such as Tekton's new events controller.
- **Reappearance.** No instance reappeared (0).

Example a reader can verify. In argo-cd, between v2.5.22 and v2.9.22, `manifests/install.yaml` drops the egress rule from the `argocd-redis` NetworkPolicy. In the same release range it adds a Role that lets argocd-redis read secrets. kube-score and kube-linter flag both changes as *introduced* security smells on existing objects. Commit tracing shows both come from fixes backported to the 2.9 branch: Redis authentication for advisory GHSA-9766-5277-j5hr, which reads the password from a secret, and a fix that removed the egress policy (PR 18382). An introduced smell is not always a careless change. In the opposite direction, microservices-demo v0.10.7 replaces the `default` service account of `shippingservice` with a dedicated one, a genuine fix that kube-linter reports as *removed* while the object stays.

Analyzer overlap. Table III compares the tools before de-duplication. Each (object, category) pair counts once over all tags. It is *both* when the two tools flagged that object in that category at some sampled tag, possibly through different checks. They agree on 134 of 241 pairs (55.6%), and on 124 of 158 (78.5%) without hygiene. The disagreement is structured by category:

- hygiene is almost entirely kube-linter's, because kube-score has no counterpart to its `use-namespace` and dangling-reference checks;
- image smells are partly kube-score-only, because only it checks pull policy;
- security and resource smells mostly agree.

This echoes [6]. It also means the lifecycle results are reported per tool, not merged.

Manual spot-check (toward RQ3). We drew 20 instances with a fixed seed: five each of persisted, fixed, object-deleted, and introduced. For each one we read the YAML at both tags. 19 were confirmed (Table IV). The exception is a "fix" in microservices-demo where the probes moved from `exec` to `grpc`: they are still identical, but kube-score no longer compares them. This is an analyzer artifact. All 9 `pod-probes-identical` removals in microservices-demo share it (every service moved to `grpc` probes in v0.9.0), so they are not counted as fixes. No renamed objects were found in the sample. We then traced all 20 to the commit behind each transition (`data/rq3_investigations.csv`), the first RQ3 evidence.

IV. SCOPE AND LIMITATIONS

Data-quality issues found.

- **Template text skipped by mistake.** A first file filter dropped real PrometheusRules and ConfigMaps because their data contains Go-template text (`{{ $labels }}`). The filter now matches only Helm-specific actions, and has tests.
- **Unrendered kustomize in flux2.** flux2's manifests are kustomize bases and patches, which we lint unrendered. The spot-check showed that flux2's deleted objects were patch fragments, so their smells partly reflect partial objects.
- **Analyzer coverage.** kube-score scores only workload-related kinds, so the overlap is limited to those kinds.
- **Analyzer artifacts.** As the grpc-probe case shows, a smell can disappear because the analyzer stops checking it, not because it was fixed.
- **Unloadable files.** Whole files kube-linter cannot load are logged, but a file where only some documents fail to parse is not detected.
- **Symlinked manifests.** Six files per tag in Knative serving (five CRDs and a ConfigMap) are git symlinks into `vendor/` and are not analyzed. Whether they appear in the skipped-file log depends on git's `core.symlinks` setting.

Feasibility risks. Five sampled tags spaced over years can hide short-lived transitions: a smell that was removed and re-added between two sampled tags looks persisted. So zero reappearances is a lower bound, not evidence that reappearance never happens. Renamed objects would still split into a removal plus an introduction.

Changes to the proposal, and their consequences.

1) **Instance key.** The key now uses the analyzer check ID rather than the category, and leaves out the file path. Two checks on one object stay two smells, and moving an object between files keeps its history. Categories are used only for aggregation.
2) **Tag sampling.** Tags are evenly spaced over four years instead of consecutive minor releases. The last five minors spanned only four to eight months for some systems and showed almost no change.
3) **Object inventory.** We added an object inventory so that deletions are not reported as fixes. Without it, RQ2 would have counted 147 fixes instead of 39.

4) **Deferred tooling.** Helm and kustomize rendering, checkov, and Trivy (RQ4) are deferred.

Remaining work.

- Render kustomize and Helm (`kustomize build`, `helm template`).
- Add denser tags for the systems that show change.
- Trace the fixes and introductions outside the spot-check to their commits for RQ3 (four of the 30 fixes are traced so far).
- Collect the two raters' labels for the category map; the sheets and agreement script are ready.

V. Progress and Contributions

Completed:

- selection of the 7 systems with recorded exclusions;
- the collection, normalization, tracking, and overlap pipeline, with tests;
- raw data for 35 versions;
- RQ1 and RQ2 baseline tables;
- a 20-instance manual spot-check, with every instance traced to its commit;
- edge-case tests for lifecycle tracking, and two-rater category-validation sheets;
- verified references for all related work;
- fresh-clone reproduction checks on two machines, and CI that reruns the tests and analysis.

All of it is linked from the artifact README (https://github.com/Rohan-Muslekar/kube-smell-tracker). Table V lists each member's main tasks and outputs, following the roles assigned in the proposal.

TABLE V
Member contributions

Member	Main tasks and outputs
Rohan Muslekar	Repository and tag selection (criteria, exclusions, evenly spaced tags); clone-and-checkout harness; per-tag object inventory; review and merge of team changes
Ankit Suman	Pinned analyzer builds; analyzer wrapper with output validation and load-failure logging; normalization to one finding schema; CI that reruns tests and checks the analysis reproduces
Yasaminzahra Valitabar	Cross-version instance tracking and transition labels; check-to-category map with rationale; fix vs. object-deletion split; lifecycle edge-case tests; two-rater category-validation sheets and agreement script
Bayan Sayyadifar	RQ1/RQ2 baseline tables; tool-overlap analysis; 20-instance manual spot-check; commit tracing of the spot-check transitions and first RQ3 case studies
Seyedeh Rojan Razavi	Report writing; related work and verified references; README, results page; methodology, validity and reproduction documents; independent fresh-clone reproduction on WSL

References

[1] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security misconfigurations in open source Kubernetes manifests: An empirical study," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 4, pp. 1–36, 2023.

[2] T. Sharma, M. Fragkoulis, and D. Spinellis, "Does your configuration code smell?" in *Proceedings of the 13th International Conference on Mining Software Repositories (MSR)*, 2016, pp. 189–200.

[3] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 164–175.

[4] A. Zerouali, R. Opdebeeck, and C. D. Roover, "Helm charts for Kubernetes applications: Evolution, outdatedness and security risks," in *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, 2023, pp. 523–533.

[5] W. Oliveira, F. Paiva, T. E. Pereira, and J. Brunet, "A defect taxonomy for infrastructure as code: A replication study," in *2025 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2025, pp. 1–11.

[6] V. Lenarduzzi, F. Pecorelli, N. Saarimäki, S. Lujan, and F. Palomba, "A critical comparison on six static analysis tools: Detection, agreement, and precision," *Journal of Systems and Software*, vol. 198, p. 111575, 2023.

[7] "KubeLinter, v0.8.3," https://github.com/stackrox/kube-linter, StackRox.

[8] "kube-score, v1.20.0," https://github.com/zegl/kube-score.